Chapter 1 · Lesson 3 — Pure Functions & Scoring

Chapter 1 · Lesson 3 — Pure Functions & Scoring

> *Goal: write `lib/scoring.ts` — two pure functions, no React, no JSX, no DOM. By the end of this lesson the heart of the application is beating, and you haven't typed `import React` even once.*

What is a "pure function" and why do we care?

A **pure function** has two properties:

1. Given the same inputs, it always returns the same output.
2. It has no side effects — it doesn't mutate any state outside itself, doesn't write to disk, doesn't make network calls, doesn't `console.log`, doesn't read a global variable that might change.

```
// Pure
function add(a: number, b: number): number {
  return a + b;
}

// Impure (uses external state)
let counter = 0;
function increment(): number {
  counter += 1;
  return counter;
}
```

Pure functions are the part of your codebase that **survives every refactor**. UI frameworks come and go — React might be replaced by Solid in three years, you might migrate to Vue,

you might one day rewrite the whole thing as a CLI tool. But `scorePick(pick, match)` doesn't change. It's just a rule about snooker scoring expressed in code.

In our app, `scorePick` and `calculateTeamScores` are the **business logic layer**. Everything else — the components, the styles, the gradients — is the **presentation layer**. Mixing them is the most common React mistake. Keeping them separated is the senior move.

> **Interview tip.** When asked *"how do you structure a React app?"*, the seniors say *"separate pure data/logic from rendering."* If you can point to your own `lib/` folder full of pure functions and your `components/` folder of pure presentation, you've got a working answer with proof.

Walking through `scorePick`

Open `lib/scoring.ts` in the repo. The first function is six lines:

```
import type { Match } from './types';

export function scorePick(pick: string | null, match: Match | undefined): num
  if (!match || !match.winner) return null;
  if (pick === match.winner) return 3;
  return 1;
}
```

Read it. Three branches:

Branch 1: `if (!match || !match.winner) return null;`

This is the "pending" branch. Two ways the function arrives here:

- **`!match`** — there's no match object at all. This is defensive. We use it because some teams have *fewer* picks than there are slots in later rounds (e.g. a team's pick lost in R1, so their R2 pick technically points at a player who doesn't appear in R2). The match lookup returns `undefined`. We don't want to crash; we want to return `null`. **Defensive coding without sloppy coding.**
- **`!match.winner`** — the match exists but no one has won yet. The `winner` field is `undefined` (recall Lesson 2: it's an *optional* field that's literally absent on pending matches).

Return `null`. Not `0`. Lesson 1 covered this; here's the line of code that enforces it.

Branch 2: `if (pick === match.winner) return 3;`

If we reach this line, we know `match.winner` is a string (TypeScript narrowed it from the previous early-return). If the team's pick matches the winner, they get **3 points**.

Notice the `===` (strict equality). In JavaScript that means *same type, same value*. Both `pick` and `match.winner` are strings, so it's a plain string comparison.

This is also why **player names must match exactly**. If a team writes `pick: 'Zhao Xintong'` but the match record says `winner: 'Zhao  Xintong'` (with two spaces), the comparison silently fails and the team gets the wrong score. This is the kind of bug that hides for days. Most teams discover it once they accidentally save the file with a trailing space and watch their score drop by 3 points.

Branch 3: `return 1;`

The match has a winner, but the pick wasn't them. The team gets a **consolation point**. Snooker's hard; you tried.

This branch is the implicit `else`. There's no `else` keyword needed because the previous two branches both `return`. Once we get past them, the only remaining case is "match finished, pick wrong."

The whole function in one breath

> *"If the match isn't finished, return null. Otherwise, return 3 if the pick won, 1 otherwise."*

That's the entire scoring rule. **Six lines of code, infinite consequences.**

Walking through `calculateTeamScores`

This is the workhorse. Open `lib/scoring.ts` and look at the second function. We'll go piece by piece.

```
import { ROUND1_MATCHES, ROUND2_MATCHES, QF_MATCHES, SF_MATCHES, FINAL_MATCH
import type { Team, TeamScores, ScoreDetail } from './types';

export function calculateTeamScores(team: Team): TeamScores {
  let r1 = 0, r2 = 0, qf = 0, sf = 0, f = 0;
```

The function takes a `Team` and returns a `TeamScores` (the wide return shape we modeled in Lesson 2: 6 numeric totals + 5 detail arrays).

The `let` declarations initialize five running totals to zero. We'll add to them as we walk each round.

Why `let` instead of `const`? Because we're going to mutate these. `let` is "I'll reassign." `const` is "I won't." TypeScript / JS strict-mode lints you on mismatches. **Naming things accurately is part of writing readable code.**

Round 1 walkthrough

```
const r1Details: ScoreDetail[] = team.r1.map((pick, i) => {
  const pts = scorePick(pick, ROUND1_MATCHES[i]);
  r1 += pts || 0;
  return {
    match: ROUND1_MATCHES[i],
    pick,
    points: pts,
    correct: pts === null ? null : pts === 3,
  };
});
```

Five lines, four ideas:

Idea 1: `team.r1.map((pick, i) => ...)` `.map()` is JavaScript's "transform every item in this array into something else." For each `pick` (a player name string) at position `i`, run the callback and collect the results. The callback receives both the value (`pick`) and the index (`i`).

We need `i` because we're going to look up the *matching* match by position: `ROUND1_MATCHES[i]`. **The invariant from Lesson 2 in action**: the i-th pick is for the i-th match. Without that invariant, this `.map` couldn't be 5 lines — it'd need a `.find()` or a lookup table.

Idea 2: `scorePick(pick, ROUND1_MATCHES[i])` We call our pure function from earlier. It returns `3 | 1 | null`. We assign that to `pts`.

This is the only place in the whole codebase that the scoring rule is applied. **Logic cen-**

tralized, used everywhere. If the league rules change next year (e.g. "5 points for picking a final-frame thriller"), there's exactly one function to update.

Idea 3: `r1 += pts || 0;` This is a small but clever line. `pts` might be 3, 1, or `null`. JavaScript's `||` operator returns the first *truthy* value, treating `null` as falsy. So:

- `pts = 3 → r1 += 3 || 0 → r1 += 3`
- `pts = 1 → r1 += 1 || 0 → r1 += 1`
- `pts = null → r1 += null || 0 → r1 += 0`

Pending picks add nothing. The total reflects only finished matches. As Round 1 plays out, `r1` creeps up from 0 toward its final value, never spiking incorrectly.

A more "modern" version would use the nullish coalescing operator: `r1 += pts ?? 0`. The `??` only treats `null` and `undefined` as falsy (unlike `||` which also rejects `0`). For our case it doesn't matter — `pts` is never `0`. But `??` is the pedantically correct choice. The codebase uses `||`; either is acceptable.

```
return {
  match: ROUND1_MATCHES[i],
  pick,
  points: pts,
  correct: pts === null ? null : pts === 3,
};
```

Idea 4: the returned detail object Each iteration of `.map` returns one `ScoreDetail`. By the end of the `.map`, `r1Details` is an array of 16 ScoreDetail objects, one per Round 1 match.

Notice `correct: pts === null ? null : pts === 3`. This is a **ternary inside a ternary** that produces a tri-state boolean:

- If `pts === null`, `correct` is `null` (pending).
- Otherwise (`pts` is 3 or 1), `correct` is `pts === 3` — i.e. `true` if 3 points, `false` if 1.

That tri-state is what powers three-color UI cells: green for correct, red for wrong, gray for pending. Without it, "wrong" and "pending" would render the same.

Rounds 2 through Final

The pattern repeats:

```
  const r2Details: ScoreDetail[] = team.r2.map((pick, i) => {
    const pts = scorePick(pick, ROUND2_MATCHES[i]);
    r2 += pts || 0;
    return {
      match: ROUND2_MATCHES[i],
      pick,
      points: pts,
      correct: pts === null ? null : pts === 3,
    };
  });
  // ... and the same shape for qf, sf
```

Five almost-identical blocks, one per round. **Duplication-by-round** is one of the recurring patterns of this codebase.

A more architected version would parameterize over a list of rounds:

```
const ROUNDS = [
  { key: 'r1', matches: ROUND1_MATCHES },
  { key: 'r2', matches: ROUND2_MATCHES },
  // ...
] as const;
ROUNDS.forEach(({ key, matches }) => { /* shared logic */ });
```

Cleaner? Yes. But it has costs:

- The TypeScript types need to be more flexible (not all rounds have the same shape; the Final is special).
- The reader needs to load the abstract loop in their head before they can see what each round does.
- Each round is currently *grep-able* by its name — a debug-friendly property.

The codebase chose copy-paste for **5 rounds in one place**. That's a trade-off, not a mistake. **Copy-paste is forgiveable up to about 3 occurrences; beyond that you should consider abstracting.** Five round blocks is right at the edge — you could argue it either way. The author chose readability. Both choices ship.

The Final is special

```
  const finalMatch = FINAL_MATCH[0];
  const fPts = scorePick(team.final, finalMatch);
  f += fPts || 0;
  const fDetails: ScoreDetail[] = [{
    match: finalMatch,
    pick: team.final,
    points: fPts,
    correct: fPts === null ? null : fPts === 3,
  }];
```

Three reasons it's different:

1. **`team.final` is a string, not an array.** Recall Lesson 2 — there's only one Final, so we accept the asymmetry.
2. **`FINAL_MATCH[0]`** — even though there's only one Final match, we kept it in an array for consistency with the other rounds. So we index into it once.
3. **`fDetails` is wrapped in an array of one** — for consistency with the other `*Details` arrays. UI code that does `team.scores.fDetails.map(...)` works exactly like `r1Details.map(...)`. **Symmetry where the UI sees it; asymmetry where the data needs it.**

The total and the return

```
  const total = r1 + r2 + qf + sf + f;
  return { r1, r2, qf, sf, f, total, r1Details, r2Details, qfDetails, sfDetai
}
```

Sum the five round totals into `total`. Return all 11 fields. **Compute once, render five different ways** — Lesson 2's "wide return shape" pays off here. Every UI screen in the app will read from this object in different ways. None of them re-runs the scoring.

Why is the return shape so wide?

You might think *"why return both `r1` (the number) and `r1Details` (the array)? Couldn't we always derive the number from the array?"*

We could:

```
const r1 = r1Details.reduce((sum, d) => sum + (d.points || 0), 0);
```

But:

- The standings table needs the number. Computing it from the array on every render of every row would be wasteful.
- The number is what most callers want; the array is what some callers want. Pre-computing both means **every caller picks what's cheap.**
- The data is small enough (5 numbers) that the storage cost is zero.

This is **the wide-return-object pattern**: when a function naturally produces several related pieces of data and they're all going to be consumed somewhere, return them all in one object. Don't make the caller call you N times.

The whole `lib/scoring.ts`

Open the file in the repo. Compare to what we've discussed:

```
import { ROUND1_MATCHES, ROUND2_MATCHES, QF_MATCHES, SF_MATCHES, FINAL_MATCH
import type { Team, TeamScores, ScoreDetail, Match } from './types';

export function scorePick(pick: string | null, match: Match | undefined): num
  if (!match || !match.winner) return null;
  if (pick === match.winner) return 3;
  return 1;
}

export function calculateTeamScores(team: Team): TeamScores {
  let r1 = 0, r2 = 0, qf = 0, sf = 0, f = 0;

  const r1Details: ScoreDetail[] = team.r1.map((pick, i) => {
    const pts = scorePick(pick, ROUND1_MATCHES[i]);
    r1 += pts || 0;
    return { match: ROUND1_MATCHES[i], pick, points: pts, correct: pts === nu
  });
  const r2Details: ScoreDetail[] = team.r2.map((pick, i) => {
```

```
    const pts = scorePick(pick, ROUND2_MATCHES[i]);
    r2 += pts || 0;
    return { match: ROUND2_MATCHES[i], pick, points: pts, correct: pts === nu
  });
  const qfDetails: ScoreDetail[] = team.qf.map((pick, i) => {
    const pts = scorePick(pick, QF_MATCHES[i]);
    qf += pts || 0;
    return { match: QF_MATCHES[i], pick, points: pts, correct: pts === null
  });
  const sfDetails: ScoreDetail[] = team.sf.map((pick, i) => {
    const pts = scorePick(pick, SF_MATCHES[i]);
    sf += pts || 0;
    return { match: SF_MATCHES[i], pick, points: pts, correct: pts === null
  });

  const finalMatch = FINAL_MATCH[0];
  const fPts = scorePick(team.final, finalMatch);
  f += fPts || 0;
  const fDetails: ScoreDetail[] = [{
    match: finalMatch,
    pick: team.final,
    points: fPts,
    correct: fPts === null ? null : fPts === 3,
  }];

  const total = r1 + r2 + qf + sf + f;
  return { r1, r2, qf, sf, f, total, r1Details, r2Details, qfDetails, sfDetai
}
```

About 40 lines. Five round blocks plus two utility functions. **No React, no JSX, no DOM.** This file would compile and run correctly in Node.js, in Deno, in a browser console, in a Cloudflare Worker, in a serverless function — anywhere JavaScript runs.

That portability is a feature. If next year you decide to write a Discord bot that posts standings hourly, this scoring file goes into the bot unchanged. It's pure logic.

Trying it out (optional)

If you want to feel it work, paste this into a Node REPL:

```
import { scorePick } from './lib/scoring';

scorePick('Wu Yize', { id: 1, p1: 'Wu Yize', p2: 'Murphy', winner: 'Wu Yize'
scorePick('Murphy',  { id: 1, p1: 'Wu Yize', p2: 'Murphy', winner: 'Wu Yize'
scorePick('Wu Yize', { id: 1, p1: 'Wu Yize', p2: 'Murphy' });
```

That's the entire scoring engine, from a black box. Three calls, three predictable answers.

Vibe prompt you would have used

> *"Write `lib/scoring.ts`. Two pure functions. (1) `scorePick(pick: string | null, match: Match | undefined): number | null` — return `null` if `!match || !match.winner`, else 3 if `pick === match.winner`, else 1. (2) `calculateTeamScores(team: Team): TeamScores` — for each of r1/r2/qf/sf use `team.rX.map((pick, i) => scorePick(pick, ROUNDx_MATCHES[i]))`, accumulate the points (treating null as 0), and produce per-pick `ScoreDetail` records of shape `{ match, pick, points, correct }` where `correct` is `points === null ? null : points === 3`. Final is special: `team.final` is a single string against `FINAL_MATCH[0]`. Return `{ r1, r2, qf, sf, f, total, r1Details, r2Details, qfDetails, sfDetails, fDetails }`. Import `Match`, `Team`, `TeamScores`, `ScoreDetail` from `./types`. No React, no side effects."*

This is the most precise vibe prompt you'll write. Notice every type is named, every shape is given, the asymmetry of Final is called out, the imports are listed. **The LLM has nothing to invent.** A prompt this specific produces the right answer almost regardless of which model you use.

CHECK YOURSELF

1. **`r1 += pts || 0` — what would happen if you removed the `|| 0`?** Walk through the case `pts = null` step by step.
2. **`scorePick` is called from inside `calculateTeamScores.map`. Is `scorePick` still**

a pure function in that context, even though it's called inside a loop? Why or why not?

3. **Why does `calculateTeamScores` return both `r1` (the total) and `r1Details` (the array of details)?** Is one redundant? Could the UI live without one of them?
4. **The Final case is hard-coded as one extra block at the bottom of the function.** Sketch what the function would look like if `team.final` were instead a one-element array (`team.final: string[]`). Does the function become longer or shorter? Easier to read or harder?
5. **You're given a new league rule: a team that picks a player who reaches the Final but loses gets +5 bonus points (regardless of whether they were the team's Final pick). Where in this codebase would you add that logic?** Is it a change to `scorePick`, to `calculateTeamScores`, or to the data? Why?

When you've answered these, you've finished Chapter 1. Onward to **Chapter 2 — Project Setup**, where we'll finally open Next.js and turn this pile of types and functions into a running web page.